

Lab 6: Structured Data Extraction

Alignment: Domain 4 - Prompt Engineering & Structured Output Estimated time: 75 minutes Mode: Local simulation; no API key required

Objective

Extract invoice data in batches, validate syntax and semantics, retry with field-level feedback, and route low-confidence records to human review.

Scenario

An accounts team receives inconsistent invoice text. The workflow must produce a fixed schema, reject invalid totals, preserve source IDs, and avoid silent guessing.

Prerequisites

- Python 3.10 or newer.
- A terminal opened in this lab folder.
- No production account, network access, or secret is required.

Architecture before execution

Read the source and identify the control boundary, the evidence artifact, and the terminal condition before running it. All side effects are confined to this lab's evidence/ folder.

Procedure

1. Inspect the JSON Schema and the two invoice fixtures. ``bashpython3 -m json.tool invoice.schema.json && sed -n '1,160p' fixtures/invoices.txt`###` **2. Run the extractor with batch size two.** ``bashpython3 -B extract.py --batch-size 2`###` **3. Inspect the output manifest for validation errors, retry counts, confidence, provenance and review status.** ``bashpython3 -m json.tool evidence/extraction.json`###` **4. Compare the ambiguous invoice with its field-level retry feedback.** ``bashpython3 -B extract.py --show-feedback`###` **5. Run the verifier and save its output.** ``bashmkdir -p evidence && python3 -B verify.py | tee evidence/verification.txt``

Acceptance checks

- ☐ Every record conforms to the declared keys.
- ☐ Totals equal subtotal plus tax.
- ☐ Invalid fields receive targeted feedback and at most one retry.
- ☐ Low-confidence records are marked for human review with source provenance.

Reflection

Why is a syntactically valid JSON object still unsafe to post into an accounting system without semantic validation?

Evidence and completion

Save terminal output in evidence/verification.txt. The lab is complete when the verifier prints PASS, every acceptance check is satisfied, and your reflection identifies one design trade-off.

Troubleshooting

- Run commands from this lab folder so relative paths resolve correctly.
- Use python3 -B to prevent learner machines from creating __pycache__ artifacts.
- If a check fails, read the named failed assertion, inspect the referenced file, and rerun only after correcting the underlying design.

Clean-up

Remove only files you created outside this folder. Never store API keys, access tokens, customer data, or production logs in lab evidence.